



POLITECNICO
MILANO 1863

School of Civil, Environmental and Land Management Engineering

Master Degree in Civil Engineering
Academic year 2026-2027

ADVANCED COMPUTATIONAL MECHANICS

Python Environment Setup

Attilio Frangi, Luca Rosafalco, Matteo Torzoni
Department of Civil and Environmental Engineering

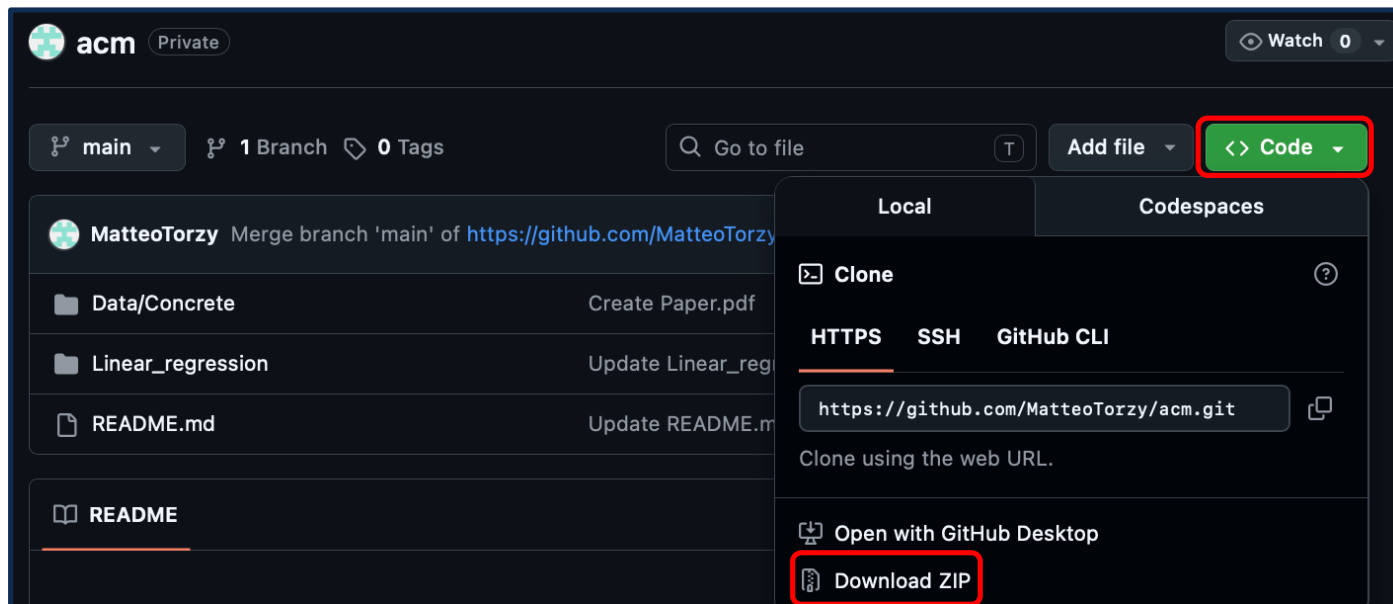
Python: Origins, Diffusion, and Packages

- Python was created at the end of the 1980s, and the first release arrived in 1991. The language was designed with a clear philosophy: code should be simple and intuitive.
- Today, Python is one of the most widely used programming languages in the world. It is extensively adopted in scientific computing, machine learning, automation, and engineering simulations.
- Python is an interpreted language, meaning that code is executed line by line by the Python interpreter rather than being compiled directly into machine code. Python scripts are usually .py files that can be executed from a terminal or an IDE such as Spyder.
- A key reason for its success is the strong open-source community that continuously develops libraries (packages), making advanced functionalities accessible with only a few lines of code.
- Packages are collections of pre-written modules and functions. They can be installed through package managers such as pip or Conda. Once installed, they are imported into a script using a import statement, allowing users to immediately access advanced functionalities.



Download the Advanced Computational Mechanics Repository

- Go to the repository page of the course: <https://github.com/MatteoTorzy/acm>
- Click the green “Code” button and choose “Download ZIP”.



- After downloading the folder, move it to the desired location.

Download and Install Anaconda

- Go to the Anaconda website: <https://www.anaconda.com/download/success?reg=skipped>
- Download the installer for your operating system (Windows, macOS, or Linux).
- Run the installer and follow the default installation steps.

Anaconda Distribution

Package and environment management with conda, Anaconda Navigator desktop app, and 600+ packages. Everything you need for data science.

GRAPHICAL INSTALLER

[↓ Windows 64-Bit Graphical Installer](#)

 Graphical install instructions

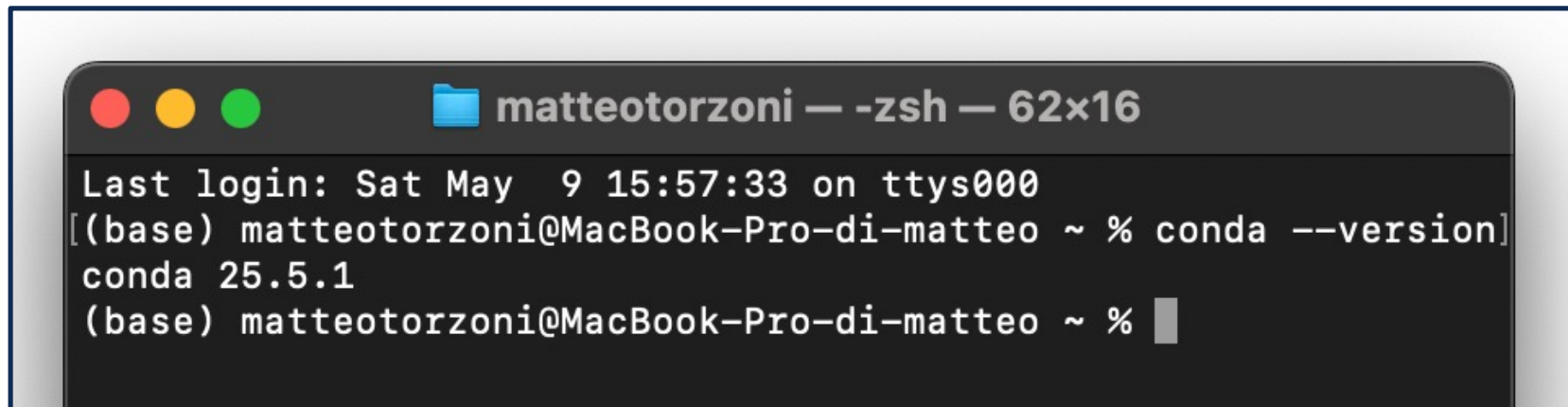
COMMAND LINE

 CLI install instructions

Verify the installation

- After installation, open Anaconda Prompt (Windows) or Terminal (macOS/Linux).
- You can verify the installation by running the following command to display the installed Conda version:

```
conda --version
```

A screenshot of a macOS Terminal window. The title bar shows the name 'matteotorzoni' and the shell '-zsh' with a resolution of '62x16'. The terminal content shows the last login time as 'Sat May 9 15:57:33 on ttys000'. The user runs the command 'conda --version' and the output is 'conda 25.5.1'. The prompt is '(base) matteotorzoni@MacBook-Pro-di-matteo ~ %'.

Create a Conda Virtual Environment

- Create a new virtual environment with a desired Python version:

```
conda create -n <myenv> python=3.12
```

- Replace <myenv> with your environment name --> "trial_env"

```
(base) matteotorzoni@MacBook-Pro-di-matteo ~ % conda create -n trial_env python=3.12
2 channel Terms of Service accepted
Retrieving notices: done
Channels:
- defaults
Platform: osx-arm64
Collecting package metadata (repodata.json): done
Solving environment: done
```

- Enter "y" when Conda asks for confirmation.

```
Proceed ([y]/n)? y

Downloading and Extracting Packages:

Preparing transaction: done
Verifying transaction: done
Executing transaction: done
```

Activate the Environment

- Activate the new environment before installing packages:

```
conda activate trial_env
```

- The terminal prompt should now show the environment name:

```
# To activate this environment, use
#
#     $ conda activate trial_env
#
# To deactivate an active environment, use
#                                     #
#     $ conda deactivate
#
(base) matteotorzoni@MacBook-Pro-di-matteo ~ % conda activate trial_env
(trial_env) matteotorzoni@MacBook-Pro-di-matteo ~ %
```

Jupyter Notebooks: Interactive Computational Documents

- Jupyter Notebook is an interactive environment that allows users to create and execute computational documents in a web browser. A notebook combines Python code, formatted text, mathematical equations, images, and plots within the same document. This makes Jupyter effective for teaching.
- The main advantage of Jupyter Notebooks is their interactivity. Code is organized into cells that can be executed independently, enabling users to test small sections of code step by step. Results such as numerical outputs, tables, and plots are displayed immediately below each cell.
- Notebooks integrate with Python scientific libraries and are compatible with Conda environments.



Install Jupyter Notebook

- Inside the (current) activated environment, install Jupyter Notebook:

```
conda install notebook
```

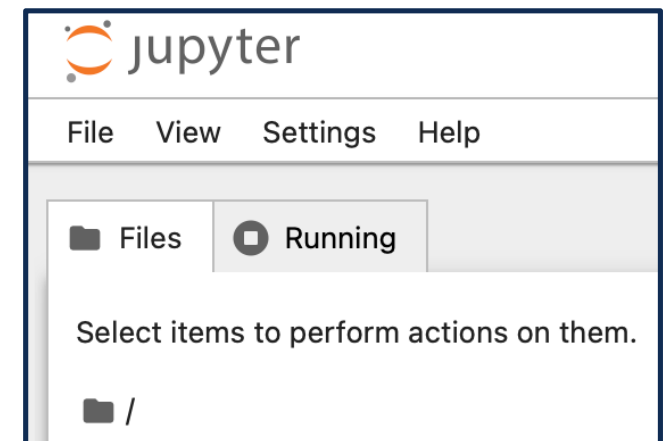
```
(base) matteotorzoni@MacBook-Pro-di-matteo ~ % conda activate trial_env  
(trial_env) matteotorzoni@MacBook-Pro-di-matteo ~ % conda install notebook
```

- Launch Jupyter Notebook using:

```
jupyter notebook
```

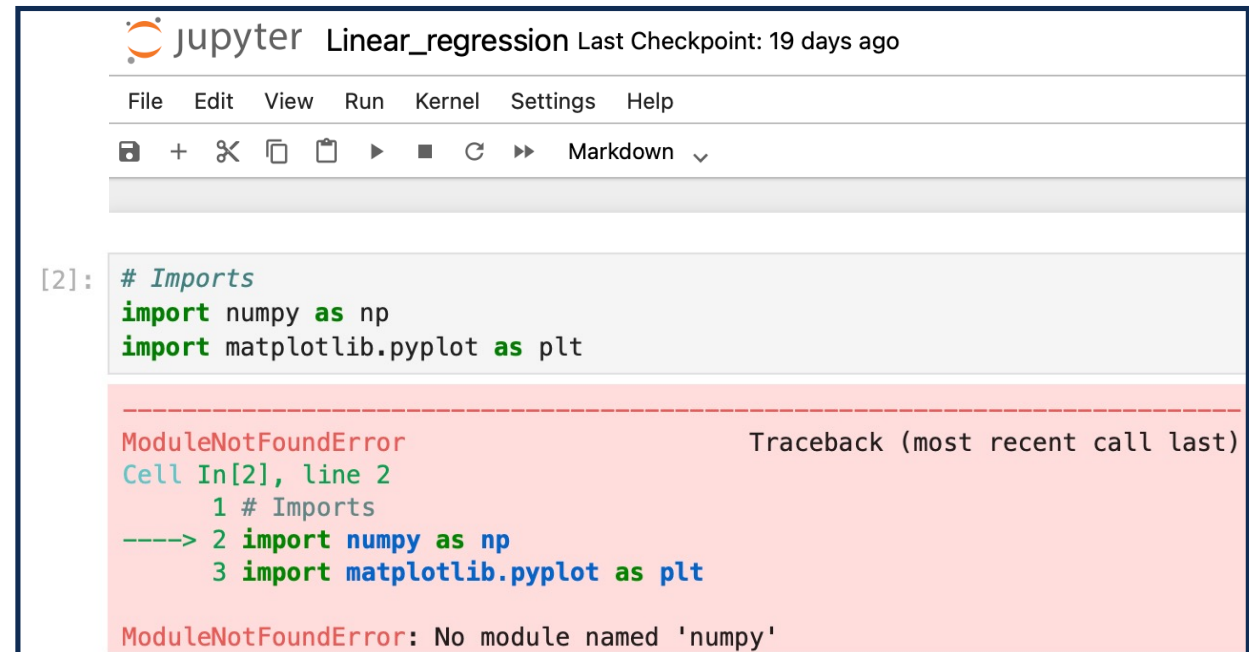
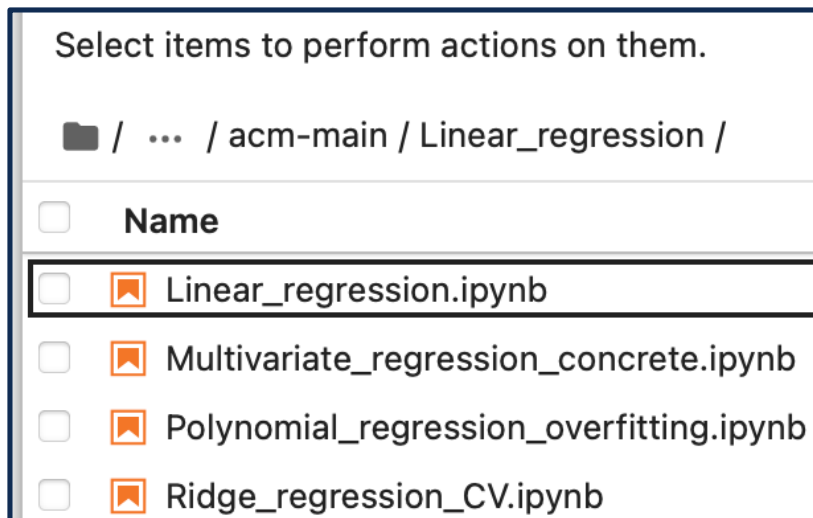
```
Proceed ([y]/n)? y  
  
Downloading and Extracting Packages:  
  
Preparing transaction: done  
Verifying transaction: done  
Executing transaction: done  
[(trial_env) matteotorzoni@MacBook-Pro-di-matteo ~ % jupyter notebook
```

- Jupyter Notebook should open automatically 



Run Jupyter Notebooks

- Open the .ipynb notebook file in your browser.
- Run notebook cells using Shift + Enter.



- Missing modules/packages can be installed as usual:

```
conda install numpy matplotlib
```

Spyder: Scientific Python Development Environment

- Spyder is an open-source Integrated Development Environment (IDE) for scientific programming with Python. It combines a text editor with tools for code execution and debugging.
- Spyder features an interface similar to MATLAB, which makes it easy to adopt for users coming from engineering backgrounds. The IDE includes a code editor, an integrated console for interactive execution, a variable explorer to inspect data structures in memory, and plotting tools for visualization.
- When installed inside a Conda environment, Spyder can directly access all the packages available in that environment, allowing users to work with libraries such as NumPy, SciPy, and Matplotlib.



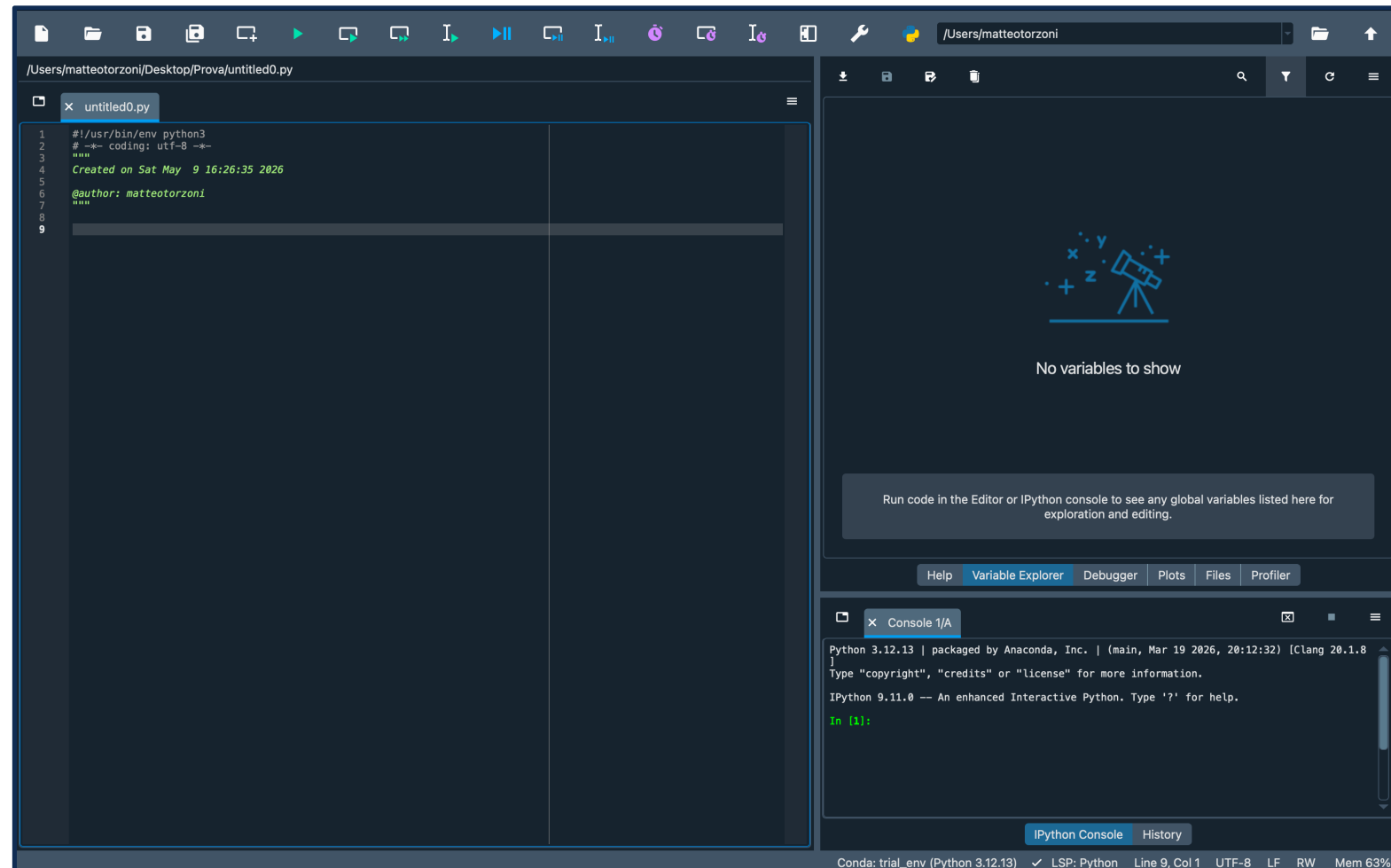
Install Spyder IDE

- Install the Spyder IDE inside the environment:

```
conda install spyder
```

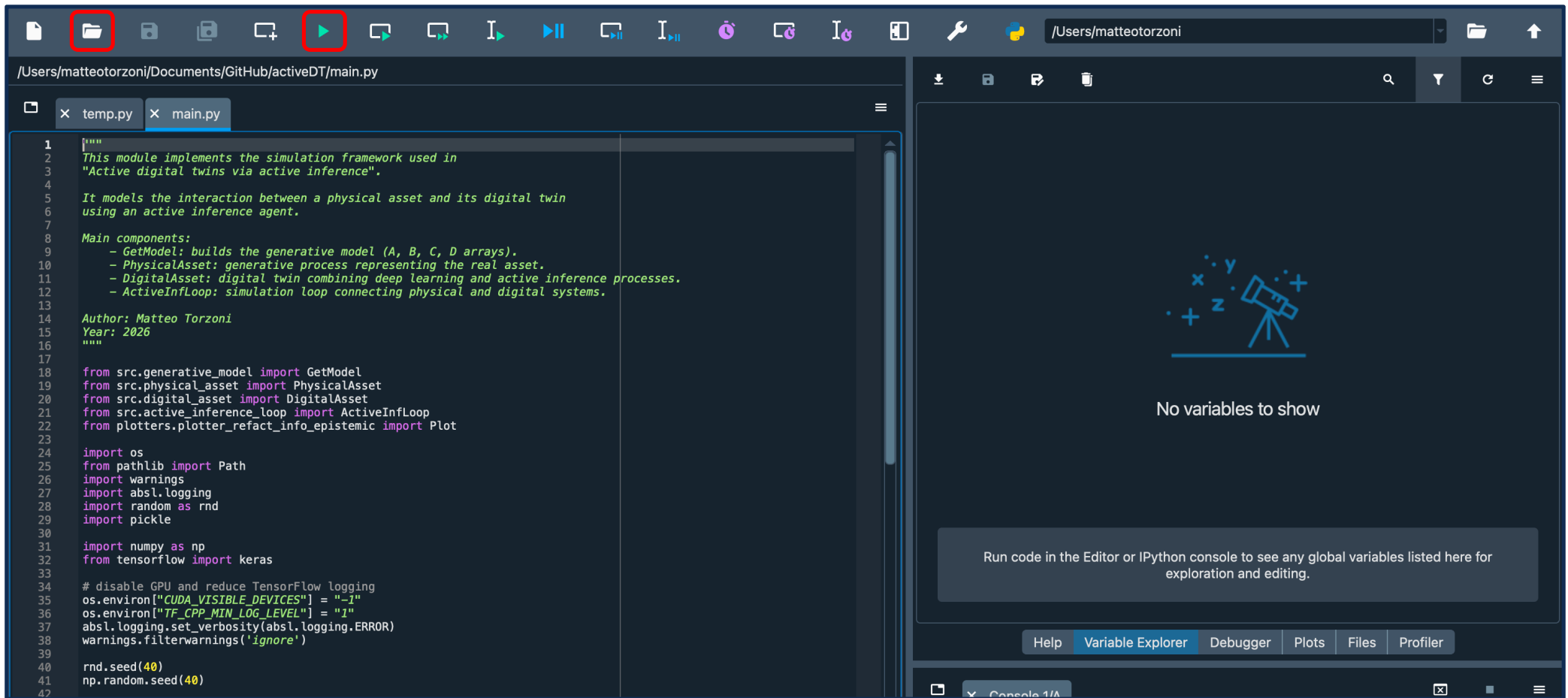
- Launch Spyder using:

```
spyder
```



Run Python scripts

- Open scripts directly in spyder to edit and run them.



- List all Conda environments:
`conda env list`
- List all packages installed in the current (activated) environment:
`conda list`
- Deactivate the current environment:
`conda deactivate`
- Install specific versions of a package:
`conda install <pkg>=<version>`
- Install additional packages using pip (if package unavailable in Conda)
`pip install <package_name>`
- Delete a virtual environment:
`conda remove --name <myenv> --all`
- Clone a virtual environment:
`conda create --name <newenv> --clone <oldenv>`

Required Packages

- notebook
- numpy
- matplotlib
- pandas
- xlrd
- scikit-learn
- spyder
- torch

Nvidia GPU setup

- Create and activate new virtual environment:

```
conda create -n trial_env python=3.12
conda activate trial_env
```

- Install CUDA driver (version 12.4) and GPU-enabled pytorch:

```
pip install torch --index-url https://download.pytorch.org/whl/cu124
```

- Install other packages as usual:

```
pip install notebook numpy matplotlib pandas xlrd scikit-learn spyder
```

- Verify GPU availability from spyder console:

```
import torch
print(torch.__version__)
print(torch.version.cuda)
print(torch.cuda.device_count())
print(torch.cuda.get_device_name(0))
```



Have a nice course!